

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ

к лабораторной работе №3

на тему:

«Стандартные типы данных, коллекции, функции, модули»

БГУИР КП 6-05 0612 02 060 ПЗ

Выполнила студентка группы 453502
КРУЧЁНОК Александра Сергеевна

Проверил канд. т. н., доцент
ЖВАКИНА Анна Васильевна

(дата, подпись преподавателя)

Минск 2026

Цель: освоить базовый синтаксис языка Python, приобрести навыки работы со стандартными типами данных, коллекциями, функциями, модулями и закрепить их на примере разработки интерактивных приложений.

Вариант 13

Ссылки:

1. <https://docs.python.org/> - офф документация (англ)
2. <https://pythonworld.ru/> - для начинающих (рус)
3. <https://smartiqa.ru/courses/python/lesson-1> - смартика (годно)
4. <https://pythonru.com/uroki/vvedenie-uroki-po-python-dlja-nachinajushhih> - краткий питон (рус)
5. <https://pythontutor.ru/> - тренажер
6. <https://pythonchik.ru/osnovy> - учебник (рус)
7. <https://younglinux.info/python/course> - онлайн-учебник (рус)
8. <https://www.w3schools.com/python/default.asp> - тесты + теория (англ)

Для защиты ЛР необходимо оформить Отчет со скринами кода и результатов его выполнения

Требования к выполнению

1. Программа должна быть снабжена комментариями на английском языке, в которых необходимо указать краткое предназначение программы, номер лабораторной работы и название, версию программы, Ф.И.О. разработчика и дату разработки.
2. Программа должна быть снабжена дружелюбным и интуитивно понятным интерфейсом
3. Выполнить документирование кода для получения справки по каждой функции
4. Каждое задание оформить в виде отдельной бизнес-функции.
5. При разработке программ рекомендуется придерживаться принципа: за решение одной конкретной задачи должна отвечать одна функция.
6. Все функции необходимо сгруппировать в модулях, согласно их логике их работы.
7. Разработанные основные функции, размещенные в отдельных модулях, нужно подключить в другом модуле, где будет происходить тестирование данных функций.
8. Размерность списка задается пользователем.
9. Предусмотреть способы инициализации последовательности: с помощью **функции генератора** и пользовательского ввода. Оформить способы инициализации в виде отдельных функций, которые на вход принимают последовательность для инициализации, и сгруппировать эти функции в отдельный модуль от основной функции программы.
10. Продемонстрировать использование **декоратора** в любом из заданий
11. В программах предусмотреть возможность повторного выполнения без выхода из программы и защиту от ввода некорректных пользовательских данных. Для этих целей рекомендуется разработать отдельные функции.
12. Обеспечить обработку конкретных классов исключений

Установка Python на Windows и практика

Download Windows installer (64-bit)

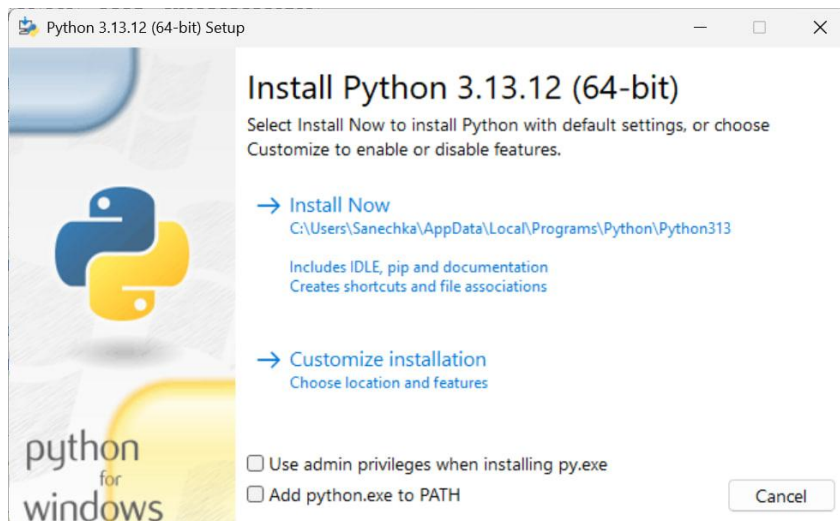
Stable Releases

- [Python install manager 26.1 - March 31, 2026](#)
 - Download [Installer \(MSIX\)](#)
 - Download [MSI package](#)
- [Python install manager 26.0 - Feb. 23, 2026](#)
 - Download [Installer \(MSIX\)](#)
 - Download [MSI package](#)
- [Python 3.13.12 - Feb. 3, 2026](#)

Note that Python 3.13.12 cannot be used on Windows 7 or earlier.

 - Download [Windows installer \(64-bit\)](#)
 - Download [Windows installer \(32-bit\)](#)
 - Download [Windows installer \(ARM64\)](#)
 - Download [Windows embeddable package \(64-bit\)](#)

Install now



Win+R, cmd -> проверка установки и работы интерпретатора в терминале (интерактивный режим >>>)
ИЛИ `py --version` , `py -m pip --version`

```
C:\Users\Sanechka>python --version
Python 3.13.12

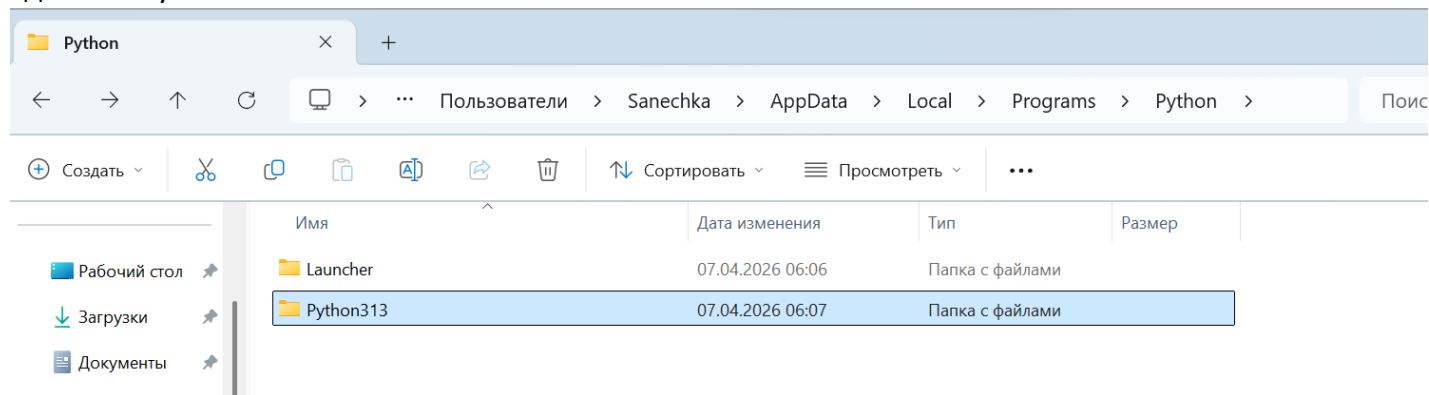
C:\Users\Sanechka>pip --version
pip 25.3 from C:\Users\Sanechka\AppData\Local\Programs\Python\Python313\Lib\site-packages\pip (python 3.13)

C:\Users\Sanechka>
```

```
C:\WINDOWS\system32\cmd.exe
Microsoft Windows [Version 10.0.26100.8037]
(c) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

C:\Users\Sanechka>python
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> print("Hello, World!")
Hello, World!
>>> exit
```

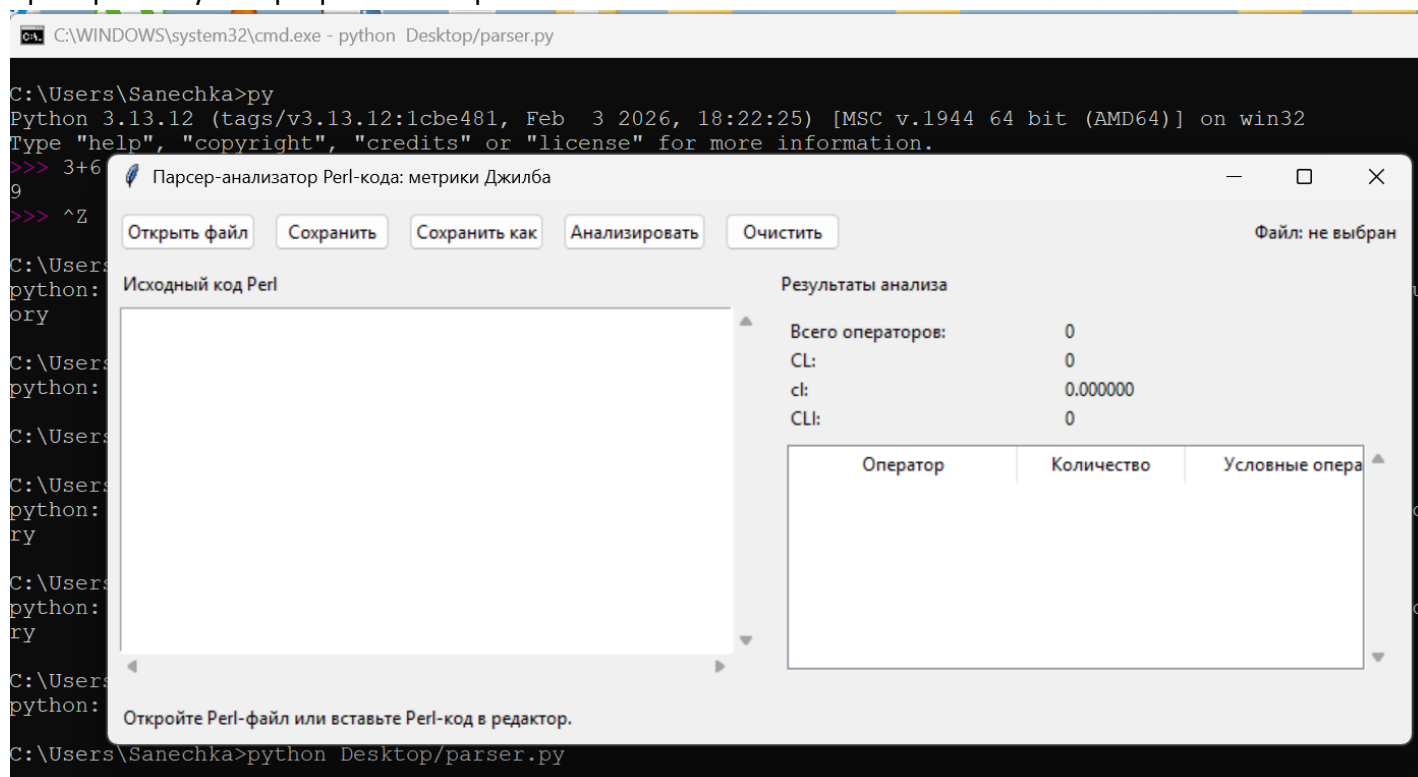
Где лежит установленный питон



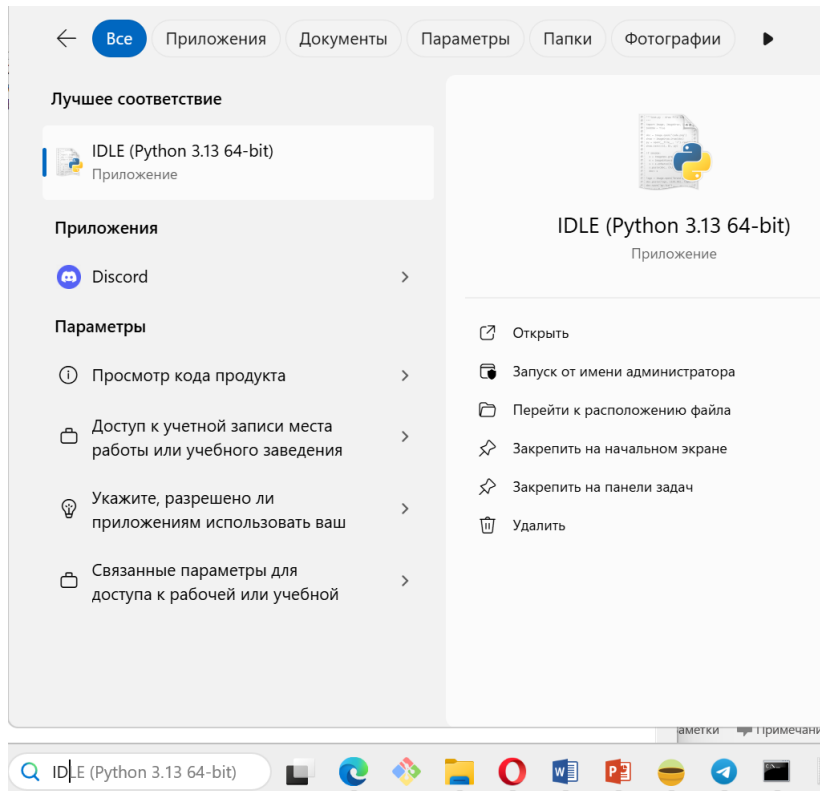
проверка и работы интерпретатора в терминале (интерактивный режим >>>)

```
C:\Users\Sanechka>py
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb  3 2026, 18:22:25) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> 3+6
9
>>> ^Z
```

проверка запуска программы в терминале



поиск IDLE



работа в IDLE

```
IDLE Shell 3.13.12
File Edit Shell Debug Options Window Help
Python 3.13.12 (tags/v3.13.12:1cbe481, Feb 3 2026, 18:22:25) [MSC v.1944 64 bit
(AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>> print ("Hello")
Hello
>>> 9+1
10
>>> import this
The Zen of Python, by Tim Peters

Beautiful is better than ugly.
Explicit is better than implicit.
Simple is better than complex.
Complex is better than complicated.
Flat is better than nested.
Sparse is better than dense.
Readability counts.
Special cases aren't special enough to break the rules.
Although practicality beats purity.
Errors should never pass silently.
Unless explicitly silenced.
In the face of ambiguity, refuse the temptation to guess.
There should be one-- and preferably only one --obvious way to do it.
Although that way may not be obvious at first unless you're Dutch.
Now is better than never.
Although never is often better than *right* now.
If the implementation is hard to explain, it's a bad idea.
If the implementation is easy to explain, it may be a good idea.
Namespaces are one honking great idea -- let's do more of those!
>>>
```

Установка линтера flake8

```
C:\Users\Sanechka>python -m pip install flake8
Collecting flake8
  Downloading flake8-7.3.0-py2.py3-none-any.whl.metadata (3.8 kB)
Collecting mccabe<0.8.0,>=0.7.0 (from flake8)
  Downloading mccabe-0.7.0-py2.py3-none-any.whl.metadata (5.0 kB)
Collecting pycodestyle<2.15.0,>=2.14.0 (from flake8)
  Downloading pycodestyle-2.14.0-py2.py3-none-any.whl.metadata (4.5 kB)
Collecting pyflakes<3.5.0,>=3.4.0 (from flake8)
  Downloading pyflakes-3.4.0-py2.py3-none-any.whl.metadata (3.5 kB)
Downloading flake8-7.3.0-py2.py3-none-any.whl (57 kB)
Downloading mccabe-0.7.0-py2.py3-none-any.whl (7.3 kB)
Downloading pycodestyle-2.14.0-py2.py3-none-any.whl (31 kB)
Downloading pyflakes-3.4.0-py2.py3-none-any.whl (63 kB)
Installing collected packages: pyflakes, pycodestyle, mccabe, flake8
Successfully installed flake8-7.3.0 mccabe-0.7.0 pycodestyle-2.14.0 pyflakes-3.4.0

[notice] A new release of pip is available: 25.3 -> 26.0.1
[notice] To update, run: python.exe -m pip install --upgrade pip

C:\Users\Sanechka>
C:\Users\Sanechka>
```

Установка доп плагина – проверки комментариев flake8

```
C:\Users\Sanechka>python -m pip install flake8-coding
Collecting flake8-coding
  Downloading flake8_coding-1.3.2-py2.py3-none-any.whl.metadata (3.1 kB)
Requirement already satisfied: flake8 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flake8-coding) (7.3.0)
Requirement already satisfied: mccabe<0.8.0,>=0.7.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flake8-coding) (0.7.0)
Requirement already satisfied: pycodestyle<2.15.0,>=2.14.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flake8-coding) (2.14.0)
Requirement already satisfied: pyflakes<3.5.0,>=3.4.0 in .\AppData\Local\Programs\Python\Python313\Lib\site-packages (from flake8-coding) (3.4.0)
Downloading flake8_coding-1.3.2-py2.py3-none-any.whl (7.6 kB)
Installing collected packages: flake8-coding
Successfully installed flake8-coding-1.3.2

C:\Users\Sanechka>
```

Использование flake8 (1 – номер строки, 32 – номер символа в строке)

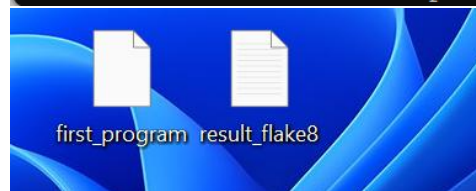
```
C:\Users\Sanechka>cd Desktop

C:\Users\Sanechka\Desktop>flake8 first_program.py
first_program.py:1:32: E999 SyntaxError: invalid decimal literal

C:\Users\Sanechka\Desktop>_
```

Записать результаты проверки flake8 в файл:

```
C:\Users\Sanechka\Desktop>flake8 first_program.py> result_flake8.txt
```



Дополнительные функции для утилиты для более подробного вывода:

flake8 --statistics --count, где --statistics это подсчет ошибок по категориям, а --count - их общие кол-во

```
C:\Users\Sanechka\Desktop>flake8 --statistics --count first_program.py
first_program.py:1:32: E999 SyntaxError: invalid decimal literal
1      E999 SyntaxError: invalid decimal literal
1

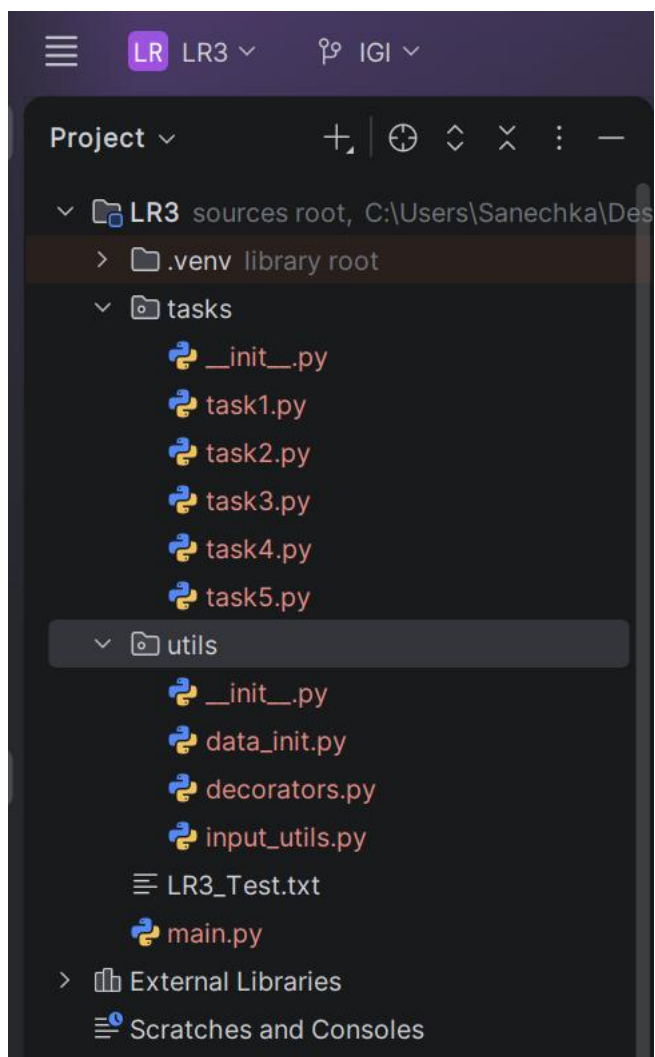
C:\Users\Sanechka\Desktop>_
```

Индивидуальные задания (вариант 13)

*предварительно для выполнения лабораторной был установлен PyCharm



Структура проекта



Задание 1. В соответствии с заданием своего варианта составить программу для вычисления значения функции с помощью разложения функции в степенной ряд. Задать точность вычислений ϵ .

Предусмотреть максимальное количество итераций, равное 500.

Вывести количество членов ряда, необходимых для достижения указанной точности вычислений. Результат получить в виде:

x	n	$F(x)$	$Math F(x)$	ϵ

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество просуммированных членов ряда, $Math F(x)$ – значение функции, вычисленное с помощью модуля `math`.

Вариант	Условие
13	$\ln(1+x) = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n} = x - \frac{x^2}{2} + \frac{x^3}{3} + \dots, x < 1$

The screenshot shows a code editor with a project named 'LR3'. The project structure includes a 'tasks' directory with files 'task1.py' through 'task5.py', and a 'utils' directory with files 'utils.py', 'data_init.py', 'decorators.py', 'input_utils.py', and 'LR3_Test.txt'. The 'task1.py' file is open, showing the following code:

```

1 """Business logic for task 1: power series calculation.
2
3 Lab 3
4 Title: Standard Data Types, Collections, Functions, and Modules
5 Version: 1.1
6 Developer: Kruchonok Aleksandra Sergeevna
7 Development date: 2026-03-29
8 """
9
10 import math
11 from collections.abc import Callable
12 from functools import wraps
13 from typing import ParamSpec, TypeVar
14
15 from utils.decorators import log_execution
16 from utils.input_utils import ask_float, ask_positive_float, ask_yes_no
17
18 MAX_ITERATIONS = 500
19 P = ParamSpec("P")
20 R = TypeVar("R")
21
22
23
24 def table_header_decorator(func: Callable[P, R]) -> Callable[P, R]: 1 usage
25     """Print table headers before the decorated result function.
26
27     Args:
28         func: Wrapped function.

```


Весь код задания 1:

```
"""Business logic for task 1: power series calculation.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1.1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-29
"""

import math
from collections.abc import Callable
from functools import wraps
from typing import ParamSpec, TypeVar

from utils.decorators import log_execution
from utils.input_utils import ask_float, ask_positive_float, ask_yes_no

MAX_ITERATIONS = 500
P = ParamSpec("P")
R = TypeVar("R")

def table_header_decorator(func: Callable[P, R]) -> Callable[P, R]:
    """Print table headers before the decorated result function.

    Args:
        func: Wrapped function.

    Returns:
        A wrapper that prints the table header before calling func.
    """

    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        print("\nResult:")
        print("-" * 70)
        print(f"{'x':>12} {'n':>8} {'F(x)':>18} {'Math F(x)':>18} {'eps':>12}")
        print("-" * 70)
        return func(*args, **kwargs)

    return wrapper

def is_valid_x(x: float) -> bool:
    """Check whether x belongs to the convergence interval.

    Args:
        x: Function argument.

    Returns:
        True if  $-1 < x < 1$ , otherwise False.
    """
    return -1 < x < 1

def calculate_ln_series(
    x: float,
    eps: float,
    max_iterations: int = MAX_ITERATIONS,
) -> tuple[float, int]:
```

```

"""Calculate ln(1 + x) using the power series.

The series is valid for |x| < 1:
     $\ln(1 + x) = x - x^2 / 2 + x^3 / 3 - \dots$ 

Args:
    x: Function argument.
    eps: Required calculation precision.
    max_iterations: Maximum allowed number of terms.

Returns:
    A tuple with the series value and the number of used terms.

Raises:
    ValueError: If x is outside the convergence interval or eps is invalid.
    OverflowError: If the required precision is not reached in time.
    """
if not is_valid_x(x):
    raise ValueError("For this series, x must satisfy -1 < x < 1.")
if eps <= 0:
    raise ValueError("eps must be positive.")

term = x
series_sum = 0.0
n = 0

while abs(term) >= eps and n < max_iterations:
    series_sum += term
    n += 1
    term *= -x * n / (n + 1)

if abs(term) >= eps:
    raise OverflowError(
        "Failed to reach the required precision "
        f"in {max_iterations} iterations."
    )

return series_sum, n

@table_header_decorator
def print_result_row(
    x: float,
    terms_count: int,
    series_value: float,
    math_value: float,
    eps: float,
) -> None:
    """Print the calculation result row in tabular format.

    Args:
        x: Function argument.
        terms_count: Number of used terms.
        series_value: Value obtained from the power series.
        math_value: Value obtained using the math module.
        eps: Required calculation precision.
    """
    print(
        f"{x:>12.6f} {terms_count:>8} "
        f"{series_value:>18.10f} {math_value:>18.10f} {eps:>12.2e}"
    )

```

```

@log_execution
def run_task_1() -> None:
    """Run the interactive interface for task 1."""
    while True:
        print("\nTask 1. Calculate ln(1 + x) using the power series.")

        while True:
            x = ask_float("Enter x (-1 < x < 1): ")
            if is_valid_x(x):
                break
            print("Input error: x must satisfy -1 < x < 1.")

        eps = ask_positive_float("Enter eps (> 0): ")

        try:
            series_value, terms_count = calculate_ln_series(x, eps)
            math_value = math.log(1 + x)
            print_result_row(x, terms_count, series_value, math_value, eps)
        except OverflowError as error:
            print(f"Calculation error: {error}")

        if not ask_yes_no("\nDo you want to repeat task 1? (yes/no): "):
            break

```

Демонстрация работы задания 1 (+ обработка исключений):

```

Run main x
⏮ ⏹ ⋮
↑ ↓ ↺ ↻ 📄 🗑
Laboratory work No. 3
Standard Data Types, Collections, Functions, and Modules
=====
1. Task 1 - Power series
2. Task 2 - Count natural numbers
3. Task 3 - Check octal number
4. Task 4 - Text analysis
5. Task 5 - Float list processing
0. Exit
Choose an option: 1

-----
Running: run_task_1
-----

Task 1. Calculate ln(1 + x) using the power series.
Enter x (-1 < x < 1): 9
Input error: x must satisfy -1 < x < 1.
Enter x (-1 < x < 1): 0.007
Enter eps (> 0): -8
The number must be positive.
Enter eps (> 0): k
Invalid input. Please enter a real number.
Enter eps (> 0): 0.000001

Result:
-----
      x      n      F(x)      Math F(x)      eps
-----
    0.007000      2    0.0069755000    0.0069756137    1.00e-06

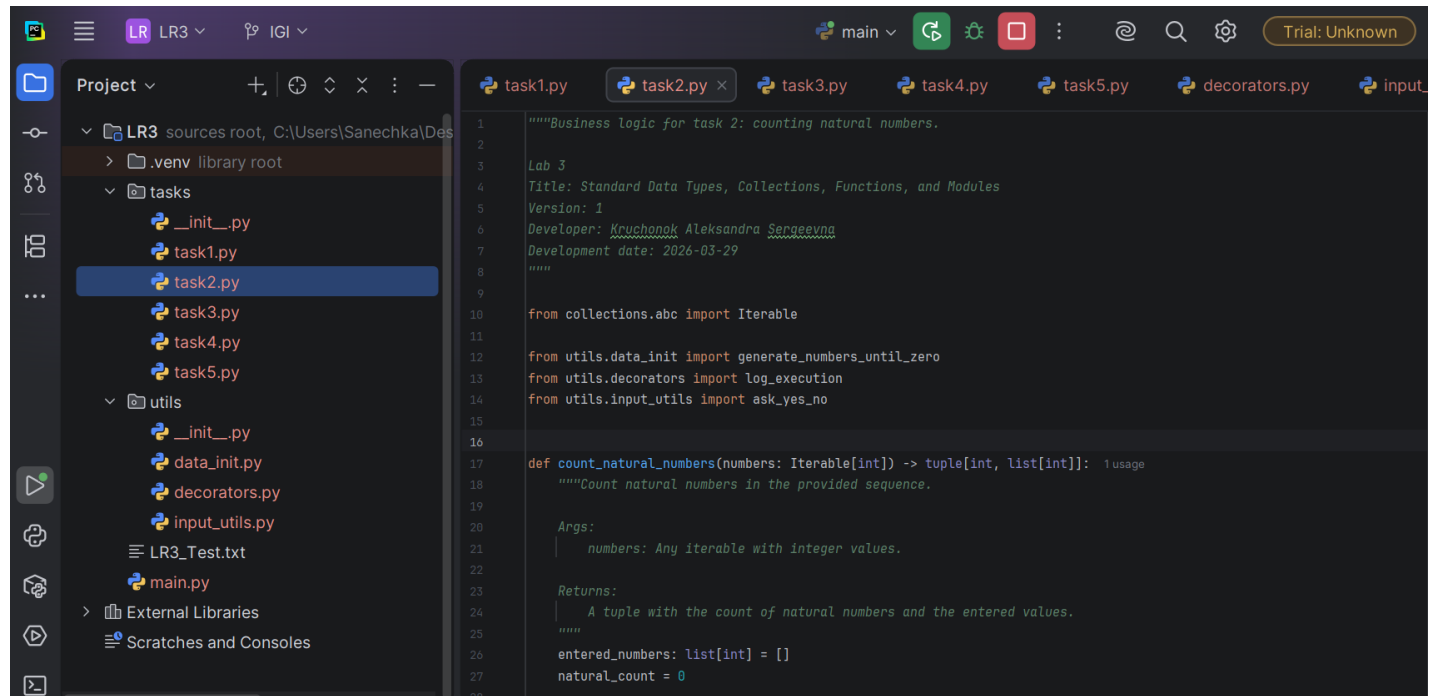
Do you want to repeat task 1? (yes/no): y

Task 1. Calculate ln(1 + x) using the power series.
Enter x (-1 < x < 1):

```

Задание 2. В соответствии с заданием своего варианта составить программу для нахождения суммы последовательности чисел.

Вариант	Условие
13	Организовать цикл, который принимает целые числа и вычисляет количество натуральных чисел. Окончание цикла – ввод 0



```
1 """Business logic for task 2: counting natural numbers.
2
3 Lab 3
4 Title: Standard Data Types, Collections, Functions, and Modules
5 Version: 1
6 Developer: Kruchonok Aleksandra Sergeevna
7 Development date: 2026-03-29
8 """
9
10 from collections.abc import Iterable
11
12 from utils.data_init import generate_numbers_until_zero
13 from utils.decorators import log_execution
14 from utils.input_utils import ask_yes_no
15
16
17 def count_natural_numbers(numbers: Iterable[int]) -> tuple[int, list[int]]: 1 usage
18     """Count natural numbers in the provided sequence.
19
20     Args:
21         numbers: Any iterable with integer values.
22
23     Returns:
24         A tuple with the count of natural numbers and the entered values.
25     """
26     entered_numbers: list[int] = []
27     natural_count = 0
28
```

Весь код задания 2:

```
"""Business logic for task 2: counting natural numbers.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-29
"""

from collections.abc import Iterable

from utils.data_init import generate_numbers_until_zero
from utils.decorators import log_execution
from utils.input_utils import ask_yes_no

def count_natural_numbers(numbers: Iterable[int]) -> tuple[int, list[int]]:
    """Count natural numbers in the provided sequence.

    Args:
        numbers: Any iterable with integer values.

    Returns:
        A tuple with the count of natural numbers and the entered values.
    """
```

```

entered_numbers: list[int] = []
natural_count = 0

for number in numbers:
    entered_numbers.append(number)
    if number > 0:
        natural_count += 1

return natural_count, entered_numbers

@log_execution
def run_task_2() -> None:
    """Run the interactive interface for task 2."""
    while True:
        print("\nTask 2. Count natural numbers in the entered sequence.")

        number_generator = generate_numbers_until_zero()
        result, numbers = count_natural_numbers(number_generator)

        print(f"Entered numbers: {numbers}")
        print(f"Count of natural numbers: {result}")

        if not ask_yes_no("\nDo you want to repeat task 2? (yes/no): "):
            break

```

Демонстрация работы задания 2 (+ обработка исключений+ неправильный ввод main):

```

Run  main x
Completed: run_task_1
-----
Laboratory work No. 3
Standard Data Types, Collections, Functions, and Modules
-----
1. Task 1 - Power series
2. Task 2 - Count natural numbers
3. Task 3 - Check octal number
4. Task 4 - Text analysis
5. Task 5 - Float list processing
0. Exit
Choose an option: 8
Please enter a value from 0 to 5.
Choose an option: 2

-----
Running: run_task_2
-----

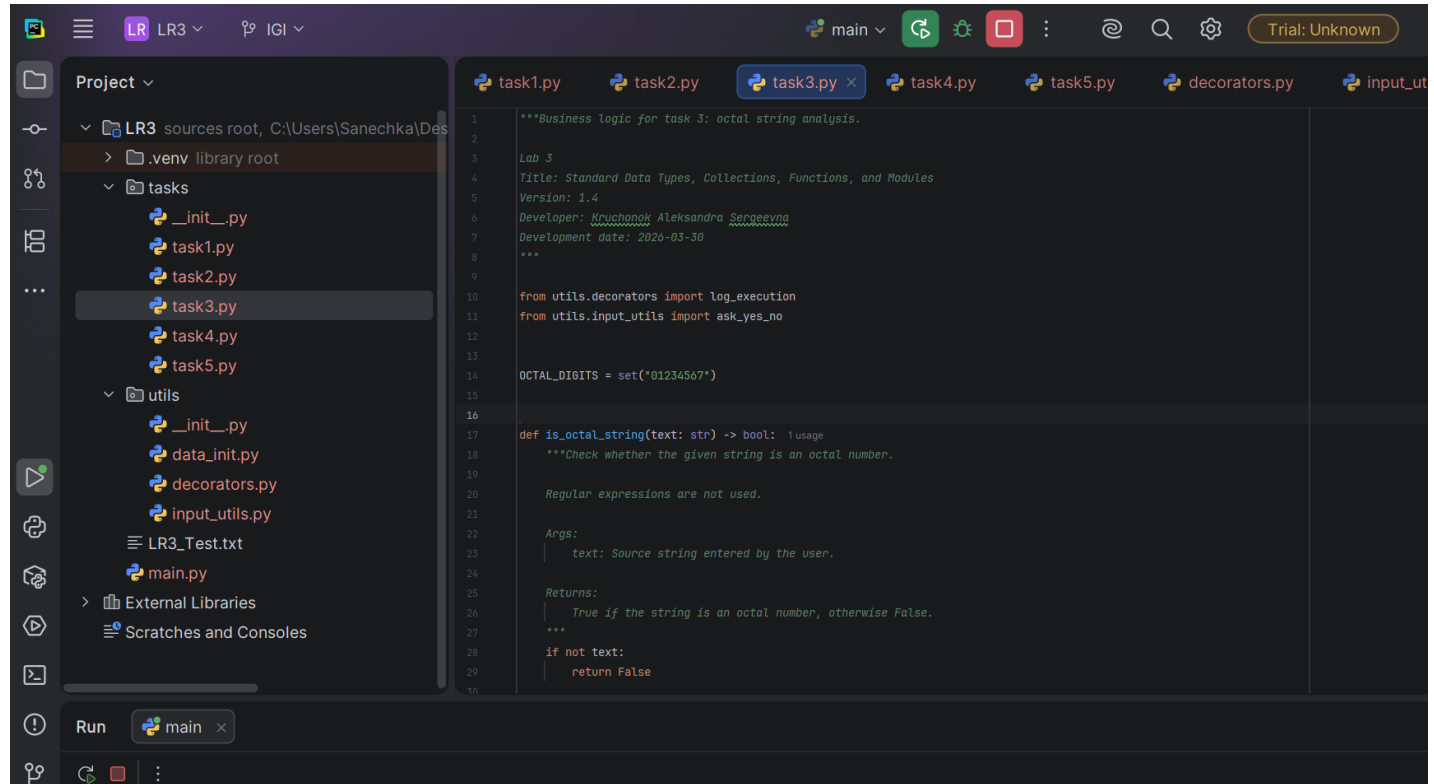
Task 2. Count natural numbers in the entered sequence.
Enter integers one by one. Enter 0 to finish.
Enter an integer: 9.9
Invalid input. Please enter an integer.
Enter an integer: 8
Enter an integer: -7
Enter an integer: -54
Enter an integer: 9877
Enter an integer: 0
Entered numbers: [8, -7, -54, 9877]
Count of natural numbers: 2

Do you want to repeat task 2? (yes/no): n
-----
Completed: run_task_2
-----

```

Задание 3. Не использовать регулярные выражения. В соответствии с заданием своего варианта составить программу для анализа текста, вводимого с клавиатуры.

Вариант	Условие
13	Определить, является ли введенная с клавиатуры строка восьмеричным числом



```
1  """Business logic for task 3: octal string analysis.
2
3  Lab 3
4  Title: Standard Data Types, Collections, Functions, and Modules
5  Version: 1.4
6  Developer: Kruchonok Aleksandra Sergeevna
7  Development date: 2026-03-30
8  """
9
10 from utils.decorators import log_execution
11 from utils.input_utils import ask_yes_no
12
13 OCTAL_DIGITS = set("01234567")
14
15
16 def is_octal_string(text: str) -> bool: 1 usage
17     """Check whether the given string is an octal number.
18
19     Regular expressions are not used.
20
21     Args:
22         text: Source string entered by the user.
23
24     Returns:
25         True if the string is an octal number, otherwise False.
26     """
27     if not text:
28         return False
29
30
```

Весь код задания 3:

```
"""Business logic for task 3: octal string analysis.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1.4
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-30
"""

from utils.decorators import log_execution
from utils.input_utils import ask_yes_no

OCTAL_DIGITS = set("01234567")

def is_octal_string(text: str) -> bool:
    """Check whether the given string is an octal number.

    Regular expressions are not used.

    Args:
```

```

    text: Source string entered by the user.

Returns:
    True if the string is an octal number, otherwise False.
"""
if not text:
    return False

return all(char in OCTAL_DIGITS for char in text)

@log_execution
def run_task_3() -> None:
    """Run the interactive interface for task 3."""
    while True:
        print("\nTask 3. Check whether the entered string is an octal number.")

        user_text = input("Enter a string: ").strip()

        if is_octal_string(user_text):
            print("The entered string is an octal number.")
        else:
            print("The entered string is NOT an octal number.")

        if not ask_yes_no("\nDo you want to repeat task 3? (yes/no): "):
            break

```

Демонстрация работы задания 3 (+ обработка исключений):

```

Run  main x
=====
Laboratory work № 3
Standard Data Types, Collections, Functions, and Modules
=====
1. Task 1 - Power series
2. Task 2 - Count natural numbers
3. Task 3 - Check octal number
4. Task 4 - Text analysis
5. Task 5 - Float list processing
0. Exit
Choose an option: 3

-----
Running: run_task_3
-----

Task 3. Check whether the entered string is an octal number.
Enter a string: 100
The entered string is an octal number.

Do you want to repeat task 3? (yes/no): y

Task 3. Check whether the entered string is an octal number.
Enter a string: 567
The entered string is an octal number.

Do you want to repeat task 3? (yes/no): y

Task 3. Check whether the entered string is an octal number.
Enter a string: 89707
The entered string is NOT an octal number.

Do you want to repeat task 3? (yes/no): y

Task 3. Check whether the entered string is an octal number.
Enter a string: Я хочу спать
The entered string is NOT an octal number.

Do you want to repeat task 3? (yes/no): r
Invalid input. Please enter yes/no.

Do you want to repeat task 3? (yes/no): n
-----
Completed: run_task_3
-----

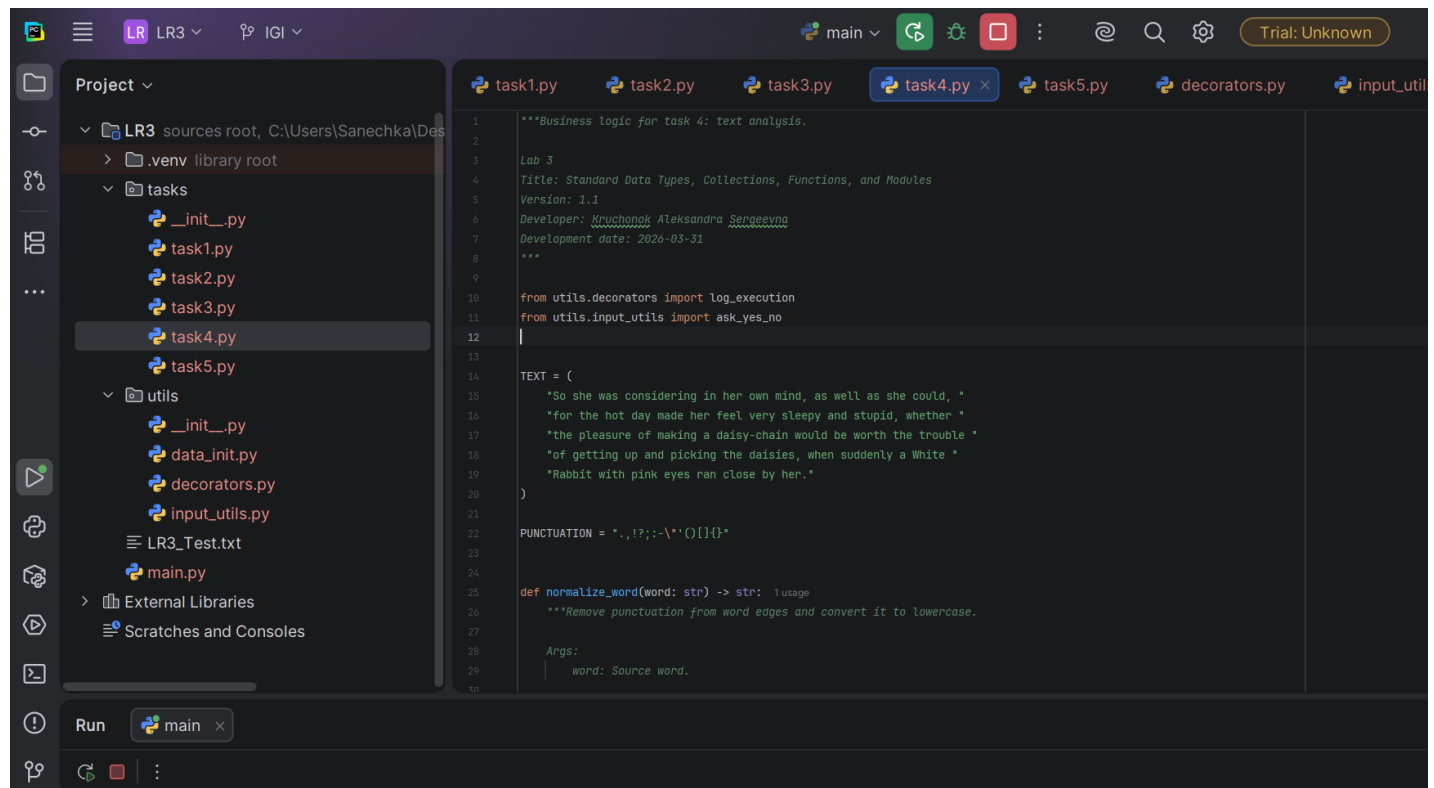
```

Задание 4. Не использовать регулярные выражения. Дана строка текста, в которой слова разделены пробелами и запятыми. В соответствии с заданием своего варианта составьте программу для анализа строки, инициализированной в коде программы:

«So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.»

Если не оговорено иное, то регистр букв при решении задачи не имеет значения.

Вариант	Условие
13	а) определить количество слов в строке и вывести на экран все слова, количество букв у которых нечетное; б) найти самое короткое слово, которое начинается на букву 'i'; в) вывести повторяющиеся слова



```
1  """Business logic for task 4: text analysis.
2
3  Lab 3
4  Title: Standard Data Types, Collections, Functions, and Modules
5  Version: 1.1
6  Developer: Kruchonok Aleksandra Sergeevna
7  Development date: 2026-03-31
8  """
9
10 from utils.decorators import log_execution
11 from utils.input_utils import ask_yes_no
12
13
14 TEXT = (
15     "So she was considering in her own mind, as well as she could, "
16     "for the hot day made her feel very sleepy and stupid, whether "
17     "the pleasure of making a daisy-chain would be worth the trouble "
18     "of getting up and picking the daisies, when suddenly a White "
19     "Rabbit with pink eyes ran close by her."
20 )
21
22 PUNCTUATION = ".,:;:-\\'(){}[]"
23
24
25 def normalize_word(word: str) -> str:
26     """Remove punctuation from word edges and convert it to lowercase.
27
28     Args:
29         word: Source word.
30     """
```


Весь код задания 4:

```
"""Business logic for task 4: text analysis.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1.1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-31
"""

from utils.decorators import log_execution
from utils.input_utils import ask_yes_no

TEXT = (
    "So she was considering in her own mind, as well as she could, "
    "for the hot day made her feel very sleepy and stupid, whether "
    "the pleasure of making a daisy-chain would be worth the trouble "
    "of getting up and picking the daisies, when suddenly a White "
    "Rabbit with pink eyes ran close by her."
)

PUNCTUATION = ". , ! ? ; : - \ " ' ( ) [ ] { } "

def normalize_word(word: str) -> str:
    """Remove punctuation from word edges and convert it to lowercase.

    Args:
        word: Source word.

    Returns:
        A cleaned lowercase word.
    """
    return word.strip(PUNCTUATION).lower()

def extract_words(text: str) -> list[str]:
    """Extract normalized words from a text without regular expressions.

    Args:
        text: Source text.

    Returns:
        A list of normalized words.
    """
    prepared_text = text.replace(",", " ")
    raw_words = prepared_text.split()

    words = []
    for raw_word in raw_words:
        cleaned_word = normalize_word(raw_word)
        if cleaned_word:
            words.append(cleaned_word)

    return words

def count_letters(word: str) -> int:
    """Count only alphabetic characters in a word.

    Args:
```

```

        word: Source word.

Returns:
    Number of alphabetic characters.
    """
    return sum(1 for char in word if char.isalpha())

def words_with_odd_length(words: list[str]) -> list[str]:
    """Find all words with an odd number of letters.

    Args:
        words: A list of words.

    Returns:
        A list of matching words.
    """
    return [word for word in words if count_letters(word) % 2 == 1]

def find_shortest_word_starting_with_i(words: list[str]) -> str | None:
    """Find the shortest word that starts with the letter 'i'.

    Args:
        words: A list of words.

    Returns:
        The shortest matching word or None if there is no such word.
    """
    matching_words = [word for word in words if word.startswith("i")]
    if not matching_words:
        return None

    return min(matching_words, key=count_letters)

def find_repeated_words(words: list[str]) -> list[str]:
    """Find repeated words in the text.

    Args:
        words: A list of words.

    Returns:
        A sorted list of unique repeated words.
    """
    counts: dict[str, int] = {}

    for word in words:
        counts[word] = counts.get(word, 0) + 1

    return sorted(word for word, count in counts.items() if count > 1)

@log_execution
def run_task_4() -> None:
    """Run the interactive interface for task 4."""
    while True:
        print("\nTask 4. Text analysis.")
        print("\nSource text:")
        print(TEXT)

        words = extract_words(TEXT)
        odd_words = words_with_odd_length(words)

```

```

shortest_i_word = find_shortest_word_starting_with_i(words)
repeated_words = find_repeated_words(words)

print("\nAnalysis results:")
print(f"Total number of words: {len(words)}")
print(f"Words with odd number of letters: {'', ' '.join(odd_words)}")
print(
    "Shortest word starting with 'i': "
    f"{shortest_i_word if shortest_i_word is not None else 'not found'}"
)
print(
    "Repeated words: "
    f"{'', ' '.join(repeated_words) if repeated_words else 'not found'}"
)

if not ask_yes_no("\nDo you want to repeat task 4? (yes/no): "):
    break

```

Демонстрация работы задания 4:

```

Run main x
=====
Laboratory work No. 3
Standard Data Types, Collections, Functions, and Modules
=====
1. Task 1 - Power series
2. Task 2 - Count natural numbers
3. Task 3 - Check octal number
4. Task 4 - Text analysis
5. Task 5 - Float list processing
0. Exit
Choose an option: 4

=====
Running: run_task_4
=====

Task 4. Text analysis.

Source text:
So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-chain would be worth the trouble of getting up and picking the daisies, when suddenly a white

Analysis results:
Total number of words: 55
Words with odd number of letters: she, was, considering, her, own, she, could, for, the, hot, day, her, and, whether, the, a, would, worth, the, trouble, getting, and, picking, the, daisies, a, white, ran, close, her
Shortest word starting with 'i': in
Repeated words: a, and, as, her, of, she, the

Do you want to repeat task 4? (yes/no): n

Completed: run_task_4
=====

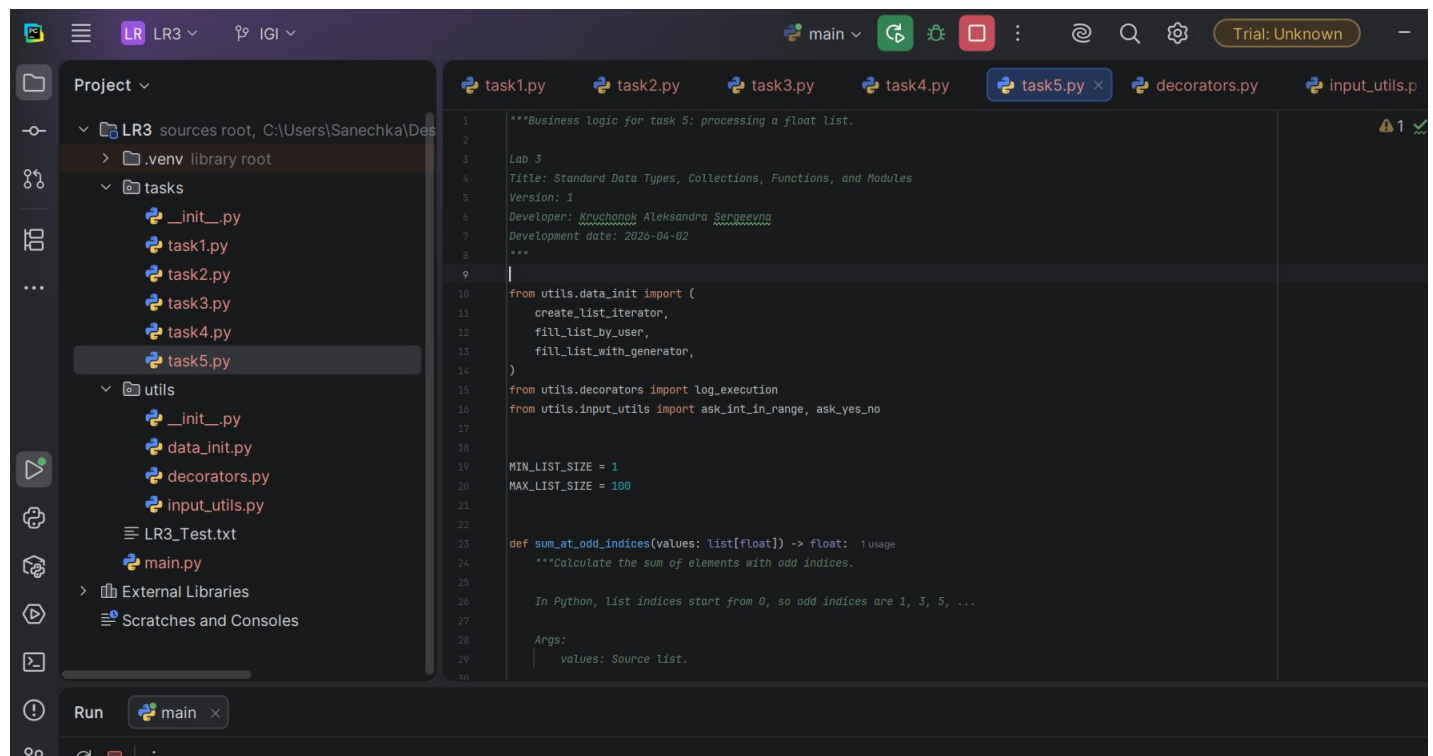
```

Задание 5. В соответствии с заданием своего варианта составить программу для обработки вещественных списков. Программа должна содержать следующие базовые функции:

- 1) ввод элементов списка пользователем;
- 2) проверка корректности вводимых данных;
- 3) реализация основного задания с выводом результатов;
- 4) вывод списка на экран.

Вариант	Условие
13	Найти сумму элементов списка с нечетными номерами и сумму элементов списка, расположенных между первым и последним отрицательными элементами

Список начинается с нулевого элемента [0] !!!



The screenshot shows an IDE with a project named 'LR3' and a file explorer on the left. The file explorer shows a directory structure with files like `__init__.py`, `task1.py`, `task2.py`, `task3.py`, `task4.py`, `task5.py`, `utils`, `data_init.py`, `decorators.py`, `input_utils.py`, `LR3_Test.txt`, and `main.py`. The main editor shows the code for `task5.py`. The code includes a docstring for 'Business logic for task 5: processing a float list', a version number, and a developer name. It also includes imports for `utils.data_init`, `utils.decorators`, and `utils.input_utils`. The code defines constants `MIN_LIST_SIZE = 1` and `MAX_LIST_SIZE = 100`. A function `sum_at_odd_indices` is defined, which takes a list of floats and returns a float. The function's docstring explains that it calculates the sum of elements with odd indices, noting that in Python, list indices start from 0, so odd indices are 1, 3, 5, ... The function's arguments are `values: Source List`.

Весь код задания 5:

```
"""Business logic for task 5: processing a float list.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-02
"""

from utils.data_init import (
    create_list_iterator,
    fill_list_by_user,
    fill_list_with_generator,
)
from utils.decorators import log_execution
from utils.input_utils import ask_int_in_range, ask_yes_no

MIN_LIST_SIZE = 1
MAX_LIST_SIZE = 100


def sum_at_odd_indices(values: list[float]) -> float:
    """Calculate the sum of elements with odd indices.

    In Python, list indices start from 0, so odd indices are 1, 3, 5, ...

    Args:
        values: Source list.

    Returns:
        The sum of elements with odd indices.
    """
    return sum(values[1::2])


def first_negative_index(values: list[float]) -> int | None:
    """Find the index of the first negative element.

    Args:
        values: Source list.

    Returns:
        The index of the first negative element or None.
    """
    for index, value in enumerate(values):
        if value < 0:
            return index
    return None


def last_negative_index(values: list[float]) -> int | None:
    """Find the index of the last negative element.

    Args:
        values: Source list.

    Returns:
        The index of the last negative element or None.
    """
    for index in range(len(values) - 1, -1, -1):
```

```

        if values[index] < 0:
            return index
    return None

def sum_between_first_and_last_negative(values: list[float]) -> float | None:
    """Calculate the sum of elements between the first and last negatives.

    The first and last negative elements themselves are not included.

    Args:
        values: Source list.

    Returns:
        The sum of elements between the first and last negative elements,
        or None if there are fewer than two negative elements.
    """
    first_index = first_negative_index(values)
    last_index = last_negative_index(values)

    if first_index is None or last_index is None or first_index == last_index:
        return None

    return sum(values[first_index + 1:last_index])

def print_list(values: list[float]) -> None:
    """Print the list in a friendly format.

    Args:
        values: Source list.
    """
    iterator = create_list_iterator(values)
    formatted_values = ", ".join(f"{value:.2f}" for value in iterator)
    print(f"List: {formatted_values}")

@log_execution
def run_task_5() -> None:
    """Run the interactive interface for task 5."""
    while True:
        print("\nTask 5. Process a float list.")

        size = ask_int_in_range(
            "Enter list size (1..100): ",
            MIN_LIST_SIZE,
            MAX_LIST_SIZE,
        )

        print("\nChoose initialization method:")
        print("1. User input")
        print("2. Generator function")

        method = ask_int_in_range("Your choice: ", 1, 2)

        if method == 1:
            values = fill_list_by_user(size)
        else:
            values = fill_list_with_generator(size)

        print_list(values)

        odd_indices_sum = sum_at_odd_indices(values)

```

```

    between_negatives_sum = sum_between_first_and_last_negative(values)

    print(f"Sum of elements with odd indices: {odd_indices_sum:.2f}")

    if between_negatives_sum is None:
        print("There are not enough negative elements for the second sum.")
    else:
        print(
            "Sum of elements between the first and last negative elements: "
            f"{between_negatives_sum:.2f}"
        )

    if not ask_yes_no("\nDo you want to repeat task 5? (yes/no): "):
        break

```

Демонстрация работы задания 5 (+ обработка исключений):

Run

main

↺

↻

⌵

⌶

⌷

🗑

Laboratory work No. 3

Standard Data Types, Collections, Functions, and Modules

=====

1. Task 1 - Power series

2. Task 2 - Count natural numbers

3. Task 3 - Check octal number

4. Task 4 - Text analysis

5. Task 5 - Float list processing

0. Exit

Choose an option: 5

Running: run_task_5

Task 5. Process a float list.

Enter list size (1..100): 4

Choose initialization method:

1. User input

2. Generator function

Your choice: 1

Enter element #1: 2.9

Enter element #2: 3

Enter element #3: -5.1

Enter element #4: 6

List: 2.90, 3.00, -5.10, 6.00

Sum of elements with odd indices: 9.00

There are not enough negative elements for the second sum.

Do you want to repeat task 5? (yes/no): y

Task 5. Process a float list.

Enter list size (1..100): 14

Choose initialization method:

1. User input

2. Generator function

Your choice: 2

List: -3.77, 2.30, -0.88, 4.28, -1.24, -2.15, 3.18, -2.83, -2.36, 0.96, 6.76, -3.72, -7.83, 4.30

Sum of elements with odd indices: 3.14

Sum of elements between the first and last negative elements: 4.30

Do you want to repeat task 5? (yes/no): n

Completed: run_task_5

Весь код файла `data_init.py` - инициализация данных, генераторы и создание итератора:

```
# Инициализация данных, генераторы и создание итератора.

"""Functions for sequence initialization.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1.1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-30
"""

import random
from collections.abc import Generator, Iterator

from utils.input_utils import ask_float, ask_int

RANDOM_MIN_VALUE = -10.0
RANDOM_MAX_VALUE = 10.0
RANDOM_PRECISION = 2

def fill_list_by_user(size: int) -> list[float]:
    """Initialize a float list using user input.

    Args:
        size: Required list size.

    Returns:
        A filled list of float values.
    """
    values: list[float] = []

    for index in range(size):
        value = ask_float(f"Enter element #{index + 1}: ")
        values.append(value)

    return values

def generate_random_float_values(size: int) -> Generator[float, None, None]:
    """Generate float values one by one using ``yield``.

    This function is a generator function. After it is called, Python creates
    a generator object, which can be used as an iterator.

    Args:
        size: Required number of generated elements.

    Yields:
        Random float values rounded to two decimal places.
    """
    for _ in range(size):
        yield round(
            random.uniform(RANDOM_MIN_VALUE, RANDOM_MAX_VALUE),
            RANDOM_PRECISION,
        )

def fill_list_with_generator(size: int) -> list[float]:
    """Initialize a float list using a generator function.
```



```

    Args:
        size: Required list size.

    Returns:
        A filled list of generated float values.
    """
    return list(generate_random_float_values(size))

def create_list_iterator(values: list[float]) -> Iterator[float]:
    """Create an iterator object for the provided list.

    Args:
        values: Source list.

    Returns:
        A list iterator object.
    """
    return iter(values)

def generate_numbers_until_zero() -> Generator[int, None, None]:
    """Generate integers entered by the user until zero is entered.

    The terminating zero is not included in the generated sequence.

    Yields:
        Entered integers except the final zero.
    """
    print("Enter integers one by one. Enter 0 to finish.")

    while True:
        number = ask_int("Enter an integer: ")
        if number == 0:
            break
        yield number

```

Весь код файла `input_utils.py` - валидация пользовательского ввода и обработка ошибок ввода:

```
# Валидация пользовательского ввода и обработка ошибок ввода.

"""Utility functions for validated user input.

Lab 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1.1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-04-01
"""

def ask_int(prompt: str) -> int:
    """Read an integer from the user with validation.

    Args:
        prompt: Text shown to the user.

    Returns:
        A valid integer.
    """
    while True:
        try:
            return int(input(prompt))
        except ValueError:
            print("Invalid input. Please enter an integer.")

def ask_float(prompt: str) -> float:
    """Read a float from the user with validation.

    Args:
        prompt: Text shown to the user.

    Returns:
        A valid float number.
    """
    while True:
        try:
            user_input = input(prompt).replace(",", ".")
            return float(user_input)
        except ValueError:
            print("Invalid input. Please enter a real number.")

def ask_positive_float(prompt: str) -> float:
    """Read a positive float from the user with validation.

    Args:
        prompt: Text shown to the user.

    Returns:
        A positive float value.
    """
    while True:
        value = ask_float(prompt)
        if value > 0:
            return value
        print("The number must be positive.")
```

```
def ask_int_in_range(prompt: str, min_value: int, max_value: int) -> int:
    """Read an integer in the specified range.

    Args:
        prompt: Text shown to the user.
        min_value: Minimum allowed value.
        max_value: Maximum allowed value.

    Returns:
        A valid integer from the specified range.
    """
    while True:
        value = ask_int(prompt)
        if min_value <= value <= max_value:
            return value
        print(f>Please enter a value from {min_value} to {max_value}.")

def ask_yes_no(prompt: str) -> bool:
    """Ask a yes/no question and return True for yes and False for no.

    Args:
        prompt: Text shown to the user.

    Returns:
        True if the user answered yes, otherwise False.
    """
    while True:
        answer = input(prompt).strip().lower()

        if answer in {"y", "yes", "д", "да"}:
            return True
        if answer in {"n", "no", "н", "нет"}:
            return False

        print("Invalid input. Please enter yes/no.")
```

Весь код файла *decorators.py* - общие декораторы:

```
# Общие декораторы.

"""Decorators used in the project.

Laboratory work No. 3
Title: Standard Data Types, Collections, Functions, and Modules
Version: 1
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-31
"""

from collections.abc import Callable
from functools import wraps
from typing import ParamSpec, TypeVar

P = ParamSpec("P")
R = TypeVar("R")

def log_execution(func: Callable[P, R]) -> Callable[P, R]:
    """Display service messages before and after function execution.

    Args:
        func: Wrapped function.

    Returns:
        A wrapper function with the same signature as the original one.
    """

    @wraps(func)
    def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
        print("\n" + "-" * 70)
        print(f"Running: {func.__name__}")
        print("-" * 70)
        result = func(*args, **kwargs)
        print("-" * 70)
        print(f"Completed: {func.__name__}")
        print("-" * 70)
        return result

    return wrapper
```

Весь код файла `main.py` - точка входа и меню:

```
"""Main module for lab 3.

Laboratory work title:
"Standard Data Types, Collections, Functions, and Modules"

Version: 2
Developer: Kruchonok Aleksandra Sergeevna
Development date: 2026-03-31
"""

from tasks.task1 import run_task_1
from tasks.task2 import run_task_2
from tasks.task3 import run_task_3
from tasks.task4 import run_task_4
from tasks.task5 import run_task_5
from utils.input_utils import ask_int_in_range

MENU_MIN_OPTION = 0
MENU_MAX_OPTION = 5


def show_menu() -> None:
    """Display the main application menu."""
    print("\n" + "=" * 70)
    print("Laboratory work No. 3")
    print("Standard Data Types, Collections, Functions, and Modules")
    print("=" * 70)
    print("1. Task 1 - Power series")
    print("2. Task 2 - Count natural numbers")
    print("3. Task 3 - Check octal number")
    print("4. Task 4 - Text analysis")
    print("5. Task 5 - Float list processing")
    print("0. Exit")


def main() -> None:
    """Start the menu-driven laboratory work application."""
    actions = {
        1: run_task_1,
        2: run_task_2,
        3: run_task_3,
        4: run_task_4,
        5: run_task_5,
    }

    while True:
        show_menu()
        choice = ask_int_in_range(
            "Choose an option: ",
            MENU_MIN_OPTION,
            MENU_MAX_OPTION,
        )

        if choice == 0:
            print("Goodbye!")
            break

        try:
            actions[choice]()
        except KeyboardInterrupt:
```

```
print("\nProgram interrupted by user. Returning to main menu.")
```

```
if __name__ == "__main__":  
    main()
```

Контрольные вопросы

1. Перечислите основные управляющие конструкции.
2. Как в языке Python реализуется механизм истинности-ложности? Может ли значение быть условием?
3. С помощью каких операторов можно комбинировать в одной условной конструкции if несколько условий? Какой механизм оптимизации применяет интерпретатор Python для эффективного вычисления результата комбинированных условных выражений?
4. Опишите синтаксис условной конструкции if-else. Представьте примерную блок-схему конструкции.
5. Опишите синтаксис условной конструкции elif. Представьте примерную блок-схему конструкции.
6. Чем использование elif будет отличаться от использования вложенных условных конструкций if-else?
7. Для чего используются циклы? Что такое итерация?
8. Какие разновидности циклов существуют?
9. Описать Python-синтаксис цикла с предусловием while.
10. Какова роль оператора break в теле цикла?
11. Какова роль оператора continue в теле цикла?
12. Какова роль оператора pass в теле цикла?
13. Может ли выражение после ключевого слова while содержать истинное значение или значение других типов данных?
14. Что такое бесконечный цикл? Когда он применяется? Привести пример кода организации диалога на тему завершения программы, либо повторного выполнения программы.
15. Если необходимо использовать вложенные циклы while для вывода элементов прямоугольной матрицы в виде строк и столбцов, какой из циклов будет печатать строки: внутренний или внешний?
16. Что такое функция? Как описывается функция в Python?
17. Зачем нужны функции?
18. Для чего используется оператор return в функциях? Как вернуть из функции несколько значений?
19. Чем формальные параметры отличаются от фактических?
20. Что такое позиционные параметры?
21. Что такое параметры по умолчанию?
22. Чем отличается глобальная переменная от локальной?
23. Чем характеризуется строковый тип данных в Python?
24. Какие есть способы объявления строк в Python?
25. Что такое неизменяемость строк?
26. Зачем нужна индексация строк, и как ее использовать?
27. Зачем и как используются срезы строк?
28. Какие основные операторы используются для работы со строками?
29. Какие основные встроенные функции класса str используются для работы со строками?
30. Приведите примеры объявления каждого из высокоуровневых
31. типов данных.
32. Объясните понятие «распаковка последовательности».
33. Какое главное отличие списков от кортежей? Когда лучше использовать кортежи, а когда – списки?
34. Как получить доступ к элементам словаря? Можно ли использовать индексацию для словарей?

35. Зачем и как используются срезы?
36. Какие операторы и встроенные функции используются для работы с кортежами, списками, словарями и множествами?
37. Какие методы есть у каждого из классов, отвечающих за каждый высокоуровневый тип данных (tuple, list, dict и set)? Опишите наиболее востребованные.